

4.4. Peripheral graphemes

4.4.1. Representing peripheral graphemes in a digital edition

Peripheral graphemes, informally also called symbols, are graphemes which we represent in transliteration by means of a placeholder grapheme dedicated to one of the basic functional classes, namely ideograms, punctuation marks, connectors and abstract symbols as defined in TG §5 and §4.4.2 below.²⁰

Since the glyph shapes and graphemic functions of peripheral graphemes only rarely show a consistent correlation across the corpora of texts we deal with, our encoding conveys information separately about the physical appearance of the glyph where applicable, and about the editor's interpretation of its function where applicable. Specifically, at the level of transliteration we use a narrow range of default placeholders (§4.4.3.1), with one placeholder dedicated to each functional class of peripheral grapheme and optionally allow arbitrary placeholders (§4.4.3.2) for greater variety and more accurate human-readable representation. The placeholders may be combined with glyph encoding (§4.4.4), which can represent the same basic functional classification as the default placeholders and, in addition, allow a fairly precise differentiation based either on the glyph's visual appearance or on its name in a source writing tradition. The synergy of transliteration and encoding is illustrated in Exhibit 4.4.A (identical to the figure in TG §5.1). Strategies for combining the available transliteration and encoding options are outlined in TG §5.1.3.

<i>Exhibit 4.4.A: representation of peripheral graphemes through transliteration and encoding</i>				
method	solution	demarcation	individuation	
			by form	by function
transliteration	default placeholders	clear	—	basic
	arbitrary placeholders	unclear when multi-character	vague	vague, implied
encoding	<g>	clear	—	—
	@type	—	—	basic
	@ref	—	granular	(implied for ideograms)

4.4.1.1. What counts as a symbol?

The symbols discussed in this section are graphemes, with the implication that they signify something that pertains, however vaguely, to language. Any other graphic content in an original document may be described in annotation, but shall not be represented in transliteration and through symbol encoding, as discussed in TG §3.1.4.3. In particular, the following features are not symbols:

- decorative or illustrative imagery, which is to be annotated
- scribal (premodern editorial) marks, which are represented in the encoding but not in the text (see §4.5.2.1 about marks used in lieu of text not inscribed; §4.5.4 about marks for scribal correction)
- as per §4.2.5 and §4.3.3, core glyphs (i.e. phonographic and numeral glyphs) are not symbols in this sense even when they are employed in a symbolic function, and shall therefore be transliterated according to their primary signification and not wrapped in glyph encoding for symbols
- however, as per TG §6.2, symbol glyphs whose shape is similar to a core glyph but not identical to that glyph as attested or expected in the given writing system (e.g. the glyph *om* written in a conspicuously archaic or stylised form) shall be treated as peripheral graphemes

²⁰ The management of symbols has changed radically from earlier DHARMA practice, so project members are requested to read this section attentively.

4.4.2. Classifying symbols by function

For the purposes of transliteration as well as encoding, we categorise peripheral graphemes into four broad functional classes: ideograms, punctuation marks, connectors and abstract symbols. These classes are discussed in TG §5 and only briefly summarised here. We find that because of the vague nature of many symbols a more precise differentiation by function would entail difficulty in defining functions and inconsistency in applying the classification. Moreover, across the writing systems in our scope — and even within a single writing tradition or a single document — any particular glyph may be used in any of these basic functions (or without a clear function). Therefore, even though particular shapes are often associated with particular functions, functional classification must be judged on a case-by-case basis, and the assignment of a grapheme to a particular function never implies any particular shape.

4.4.2.1. Ideograms

- ideograms (TG §5.2) are graphemes that signify words, morphemes or abstract concepts
- modern international examples include |+| |%| |\$| |€|
- examples in Indic writing systems include |ॐ| (Devanagari *om*), |ௐ| (Tamil *piḷḷaiyār cuḷi*)
- numerals are technically also ideograms, but from a practical point of view we treat them as core graphemes (§4.3)

4.4.2.2. Punctuation marks

- punctuation marks (TG §5.3) are graphic signs employed for segmentation into syntactic or metrical units such as sentences, clauses, list items and stanzas or their subdivisions
- throughout this Guide, the term ‘punctuation mark’ is always used in this specific and restricted sense
- thus, figural and ornamental signs, as well as “typographic” signs used to mark the end or beginning of an entire text or a major section of text, are generally excluded from this class
- modern international examples include |.| |,| |!|
- examples in Indic writing systems include vertical bars, horizontal dashes and their variations

4.4.2.3. Connectors

- connectors (TG §5.4) are a special set of marks which signify that a spatial interruption in the script is not to be understood as a break in linguistic structure
- connectors include the marks called ‘space fillers’ in previous versions of this Guide, used where a part of the writing surface could not accommodate the text to be written, filling it up with one or more marks in order to preserve the continuity of the writing
- as well as marks used singly at line ends to signify that a word break is not present (like a modern hyphen)
- signs used by the scribe as placeholders for characters of text he was unable to provide (for example because they were illegible in the manuscript being copied) are not connectors and shall be encoded as per §4.5.2.1

4.4.2.4. Abstract symbols

- abstract symbols (TG §5.5) are all peripheral graphemes that cannot be confidently assigned to either of the above three classes
- abstract symbols are also called simply ‘symbols’, but we also use the term ‘symbol’ in a generic sense to refer to all peripheral graphemes
- classifying a grapheme as an abstract symbol means choosing not to assert any particular linguistic function for it, but is not an assertion that it has no function at all
- thus, all graphemes whose function is ambiguous or uncertain shall be classified as abstract symbols, even if there is a chance that they might qualify for one of the specific classes
- encoding an abstract symbol instead of a specific function is not an error and will have little ultimate impact on the quality of our corpus

4.4.3. Transliterating peripheral graphemes

The placeholders used for peripheral graphemes are a device of transliteration, and are accordingly discussed in TG §5. Here, we summarise only the essentials. We dedicate a default placeholder to each of the broad functional classes introduced in §4.4.2 above. When an arbitrary placeholder (§4.4.3.2) is not desired, every peripheral grapheme in the source must be transliterated with the default placeholder appropriate to its class.

4.4.3.1. Default placeholders

- the default placeholders for the classes are as follows
 - for ideograms: @ (at sign)
 - for punctuation marks: . (dot, period or full stop)
 - for connectors: § (section sign)
 - for abstract symbols: * (asterisk)
- in XML editions, the default placeholders will usually be combined with glyph encoding (§4.4.4)
- default placeholders may be used without being wrapped in a <g> element in the following circumstances:
 - when supplying editorial punctuation for the purpose of semantic segmentation, as per §6.3.6
 - when encoding a text from a previous edition, without access to the original or a surrogate, if that edition records the presence of a symbol without describing its appearance

4.4.3.2. Arbitrary placeholders

At the editor’s discretion, arbitrary target graphemes (or strings of target graphemes) may be used for representing any peripheral grapheme in the source. Arbitrary placeholders can be Unicode characters chosen to resemble the source grapheme’s graphetic appearance or to imitate its perceived graphemic function, as well as strings alluding to an emic name of the source grapheme or describing it in etic terms (TG §5.1.2).

- in XML editions, it is generally preferable to use only the default placeholders and combine them with glyph encoding (§4.4.4)
- nonetheless, because they can help reduce code clutter, arbitrary placeholders are permitted in XML contexts if they are deemed to be satisfactory representations of the source grapheme
 - arbitrary placeholders may be used without glyph encoding provided that they consist of a single character
 - multi-character arbitrary placeholders must, however, always be combined with glyph encoding (§4.4.4) to make it clear that a particular string of target characters corresponds to a single source grapheme
- strategies for combining the available transliteration and encoding options are outlined in TG §5.1.3
- in XML editions, arbitrary placeholders are most likely to be used for basic forms of punctuation marks, as follows
 - | (U+007C Vertical Line) for signs comprised of a single plain vertical bar
 - / (slash) for signs comprised of a single vertical bar with a hook, crossbar or ornamental addition
 - , (comma) for short, predominantly vertical and often curved strokes at any height
- additional recommendations for arbitrary placeholders are available in the TG sections (under §5) discussing the functional classes

4.4.4. Glyph encoding for peripheral graphemes

We use the TEI element <g> (for “glyph” or “gaiji”²¹) in the encoding of all glyphs other than phonographic ones and decimal digits. The use of this element indicates that our transliterated character set provides no

²¹ <https://en.wiktionary.org/wiki/gaiji>

direct equivalent to the original glyph, and that the target character or string within this element represents a single source grapheme.

- the (default or arbitrary) placeholders that correspond in transliteration to a peripheral grapheme in the source may be wrapped in the element `<g>`
 - this encoding is mandatory for arbitrary placeholders that consist of more than one character and optional but recommended for all other placeholders
- the `<g>` element for peripheral graphemes can be supplemented with the following attributes
 - optionally or as required, `@type` to assign the grapheme to a functional class as per §4.4.4.1
 - optionally, `@ref` to associate the grapheme with an item in our authority file for symbols as per §4.4.4.2
- in DHARMA practice, `<g>` for symbol encoding shall always contain text (a transliteration placeholder)
 - unlike the earlier versions of this Guide, which endorsed empty `<g/>` elements for certain kinds of symbols

4.4.4.1. Encoding symbol function

The attribute `@type` on the `<g>` element for peripheral graphemes formally associates the glyph with one of the basic functional classes introduced in §4.4.2. It thus redundantly represents the same information as the choice of default placeholder, but does so in a rigorous machine-readable manner.

- the available values of `@type` for symbol encoding are as follows²²
 - `"ideogram"` for ideograms (other than numerals)
 - `"punctuation"` for punctuation marks
 - `"connector"` for connectors
 - `"symbol"` for abstract symbols, i.e. anything not explicitly assigned to any of the above classes
 - this is the default value of `@type`, meaning that if the attribute is not present in the code, the `<g>` element will automatically be understood to have this type
 - for this reason, `@type` should always be explicitly encoded in conjunction with default placeholders other than `*` (i.e. default placeholders for any of the other three classes), because omitting `@type` would then result in a contradictory classification

4.4.4.2. Encoding symbol identification through the authority file

The attribute `@ref` on the `<g>` element for peripheral graphemes formally associates the glyph with an entry in our authority file for Glyph Taxonomy.²³ The contents and use of the of the Taxonomy are explained in the authority file itself and are only briefly summarised here.

The authority file is a classified list of **symbol tokens**, which are short descriptive labels identifying various kinds of symbols. Within the authority file, tokens are divided at the top level into an etic and an emic class. The etic class consists of tokens that describe the graphic appearance of symbols in simple and objective terms. This class is structured hierarchically in the manner of a taxonomy, with a small number of “genus” tokens for basic classes such as vertical bars, horizontal dashes, bullets, etc., and a larger number of “species” and “subspecies” tokens for more or less specific varieties of these classes. The emic class includes tokens that identify a language and specify a source-language term for a particular symbol that has a name in a given writing tradition. The language tags in the emic tokens correspond to the genus names of the etic class, while the symbol terms correspond to species names.

For each token, the authority file includes a “name” and a “description.” Names are either very concise indications of the symbol's appearance, or terms designating the symbol's meaning. Descriptions contain more detailed, but still concise, information about the symbol. Entries also optionally include a “mapping,”

²² In addition to these values, `"numeral"` is used for numeral glyphs that cannot be represented in transliteration with a single Arabic digit, as per §4.3.1. The DHARMA schema also allows the value `"phonographic"`, which is not used in our present encoding practice, but is included as a provision for the future when it might become desirable to use encoding to classify alternative shapes (graphetic allographs) of phonographic glyphs.

²³ To view the file rendered for display, visit <https://dharmalekha.info/glyphs>. To view and, if necessary, edit the raw XML, go to DHARMA_glyphTaxonomy.xml in the project-documentation repository.

which is a Unicode character that is in some way similar to the original symbol. These details are involved in the generation of display in a DHARMA environment (§4.4.4.3).

- the value of `@ref` on the `<g>` element representing a symbol must begin with the abbreviation `"sym:"`, indicating that it refers to our Glyph Taxonomy authority file, followed by a symbol token, e.g.
 - with emic tokens, `"sym:tam-kalancu"` to refer to the sign called *kaḷañcu* in the Tamil writing tradition
 - with etic tokens, `"sym:bullet-double"` to refer to a sign in the shape of a double bullet (an entity of the “bullet” genus and within that to the “double” species)
- when `@ref` is not present on a symbol encoded with `<g>`, the symbol will by default be understood to be unclassified, meaning that the encoder has chosen not to associate this peripheral grapheme with the Taxonomy, for example
 - because the symbol is too unclear to assert any shape for it
 - because no visual documentation is available and a previous editor did not describe the symbol
 - because the shape of the symbol is not deemed relevant

4.4.4.3. Rendering symbols for display

When rendered for display in the DHARMA ecosystem, encoded symbols will appear as described below. Arbitrary placeholders used without a `<g>` wrapper will of course be shown as plain text. The examples in §4.4.4.4 also show the display rendition of encoded symbols.

- the actual text representing the symbol will be as follows:
 - by default, the mapping assigned to the token in the authority file (§4.4.4.2) will be displayed
 - this is possible whenever the symbol is encoded with a `<g>` element, which has the attribute `@ref`, which points to a valid token in the authority file, which has a “mapping”
 - by design, mapping is included in the authority file only for a limited range of symbols that can be satisfactorily mapped to reasonably similar-looking digital characters with reasonably broad font support
 - if no mapping is available for a symbol, then the name assigned to the token in the authority file (§4.4.4.2) will be displayed
 - this is possible whenever the symbol is encoded with a `<g>` element, which has the attribute `@ref`, which points to a valid token in the authority file
 - all tokens in the authority file have a name
 - the objective of this display is to indicate the precise type of the symbol to the end user when no mapping has been assigned to a token
 - if neither a mapping nor a name are available, then the content of the `<g>` element will be displayed
 - this is possible whenever the symbol is encoded with a `<g>` element
 - the objective of this display is primarily to show an arbitrary placeholder (§4.4.3.2) used in transliteration, but it is also a fallback display option for default placeholders encoded with `<g>` but not associated with a valid token (because `@ref` was not added or its value is not a valid token)
- the above text will be highlighted with a different font colour (purple)
 - when the text displayed is the name assigned to a token, then this will also be surrounded with angle brackets and a cartouche, to disambiguate the name from text present in the source and to allow retaining this disambiguation when the rendered text is copied and pasted into another working environment
- when the user moves their mouse over the display rendition of an encoded symbol, the following additional information will be shown
 - the functional class of the symbol (§4.4.2) if encoded with `@type`
 - the name of the symbol if a valid token is encoded with `@ref`
 - the description of the symbol if a valid token is encoded with `@ref`

4.4.4.4. Examples of symbol encoding

The Exhibits in this section provide examples of symbols transliterated with default placeholders and encoded for function and identification.

Exhibit 4.4.B: encoding ideograms

<code><g type="ideogram" ref="sym:tam-varusam">@</g></code>
➤ this will appear in display as (𑖦𑖩) (based on the mapping in the authority file)

Exhibit 4.4.C: encoding punctuation marks

deśam apālayaT<g type="punctuation" ref="sym:doubleBar-serifed">.</g> tat-putro jayasimhas trayastrimśataṁ<g type="punctuation" ref="sym:bar-serifed">.</g>
➤ these will appear in display as <i>apālayaT</i> and <i>trayastrimśataṁ</i> (based on the mapping in the authority file) ➤ if a rough representation of the shape is sufficient, the arbitrary placeholders and may be used ➤ encoding is not required for , but mandatory for , which consists of two characters ➤ <g type="punctuation" ref="sym:doubleBar-serifed"> </g> for full identification (displayed as , not as , as the mapping in the authority file overrides the content) ➤ <g type="punctuation"> </g> to clarify only the function (displayed as , the content of the <g> element, because there is no reference to the authority file) ➤ <g> </g> as a minimal representation (displayed as because there is no reference to the authority file) ➤ alternatively, (U+2016 Double Vertical Line), a single Unicode character, may be used without encoding

Exhibit 4.4.D: encoding connectors

karuhun di:rgḥāyūrāro<g type="connector" ref="sym:bar-curly">§</g>
alternatives: ➤ § as a minimal representation without encoding ➤ (U+00A6 Broken Bar) as an arbitrary placeholder without encoding ➤ <g type="connector">§</g> to make no assertion regarding shape (displayed as the content § because there is no @ref) ➤ <g ref="sym:bar-curly"> </g> or <g ref="sym:bar-curly">*</g> to make no assertion regarding function ➤ both of these will be displayed as (the mapping assigned to the token in the authority file overriding the content) ➤ if the default placeholder § is wrapped in <g>, then @type must also be present (§4.4.4.1), while arbitrary placeholders may be wrapped in <g> without @type

Exhibit 4.4.E: encoding abstract symbols



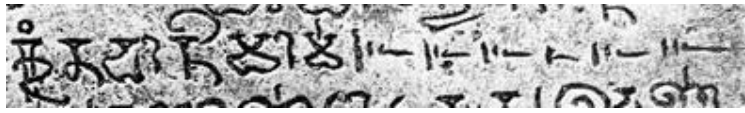
`<g ref="sym:floret-complex">*/</g> svasti`

- `type="symbol"` could be added, but is not necessary because this is the default value
- the display will be ✱ (the mapping assigned to the token in the authority file)
- alternatively, an arbitrary placeholder such as ✱ might be used, with or without a `<g>` wrapper and with or without `@ref`
 - if `@ref` is not present, then the arbitrary placeholder will be displayed

4.4.5. Multiple peripheral graphemes

- groups of peripheral graphemes appearing together, whether identical or different, shall normally be encoded with separate `<g>` wrappers
 - clusters of symbols that are frequently used together in a given writing tradition can, however, be treated as single glyphs and, accordingly, wrapped in just one `<g>` element
 - this applies in particular to the double vertical bars which are quite ubiquitous in the Indic writing system
 - but can also apply to more elaborate combinations when these belong to an established convention, such as the Balinese punctuation marks in TG §5.3.1
- series of more than two identical glyphs (e.g. three vertical bars) are best treated as combinations of singles, unless some of them are clearly doubles (as indicated by spacing or by additional strokes joining the pairs)
- the requirement of individual wrapping applies likewise to connectors (for which earlier versions of this Guide prescribed a single wrapper)
 - this increases code clutter for iterated connectors, as illustrated in Exhibit 4.4.F, but is desirable in order to ensure consistency in the encoding

Exhibit 4.4.F: encoding multiple connectors



kṛtavān imām· `<g type="connector" ref="sym:flourish-taperOut-bars">§</g><g type="connector" ref="sym:flourish-taperOut-bars">§</g><g type="connector" ref="sym:flourish-taperOut-bars">§</g><g type="connector" ref="sym:flourish-taperOut-bars">§</g><g type="connector" ref="sym:flourish-taperOut-bars">§</g><g type="connector" ref="sym:flourish-taperOut-bars">§</g>`

4.4.6. Peripheral graphemes and their context

- use spaces around transliterated symbols as instructed in TG §7.3.4.4, i.e. essentially:
 - add spaces to separate the transliterated (and, if applicable, encoded) representation of any and all peripheral graphemes from any and all adjacent graphemes, except:
 - punctuation marks shall not be separated with a space from preceding text
 - groups of symbols appearing together may be spaced or not, at your discretion
 - use no spaces around a peripheral grapheme that interrupts a word, as in Exhibit 4.4.D
- if a peripheral grapheme appears at the boundary of two block-level containers for intrinsic structure (§2), allocate it to one of the containers depending on your judgement of its semantic function
 - when in doubt, e.g. because a generic symbol marks the transition from one passage to another without being semantically associated with either, allocate it to the end of the earlier containing block
- as a special case, peripheral graphemes, especially punctuation marks, may be written at an *akṣara* boundary next to, but not coincident with, a word boundary (which would fall within the *akṣara*)

- such symbols must always be represented in the transliterated text at their actual location (and thus without editorial spacing)
- however, it is recommended that you flag this as non-standard usage and optionally also normalise the text (§6.3) by restoring the symbol at the word boundary and changing the sandhi and editorial spacing as applicable, as in Exhibit 4.4.G
- when this happens at the end of a container element such as a verse line, encode the container boundary at its logical place (§2.2.4.1), disregarding the *akṣara* boundary and the symbol

Exhibit 4.4.G: punctuation placed at *akṣara* boundary rather than word boundary



- flagged: ...yuvarājaḥ pañcaviṃśati<orig>|n</orig>tat-putro...
- normalised: ...yuvarājaḥ pañcaviṃśati<choice><orig>|n</orig><reg>M|</reg></choice>tat-putro...
- the punctuation mark belonging at the end of the word *pañcaviṃśati* is written after *ti*, because the end of this word has been inscribed as a homorganic nasal forming a conjunct with the beginning of the next word
- instructions for flagging and normalising non-standard usage are found in §6.3
- the arbitrary placeholder | used in the illustration could of course be supplemented with encoding or replaced with a default placeholder plus encoding